Scrabble Score Board specification

Contents

1. Introduction - p.2
2. Background and Goals - p.2
3. Implementation Plan - p.3
4. Functional Requirements - p.4
5. Non-Functional Requirements - p.9
6. Screen Transitions - p.10
7. Revisions - p.13

1. Introduction

This document outlines the specifications for a Scrabble score management application and provides guidelines for design decisions and implementation steps throughout the development process.

The primary intended reader of this specification is the application's developer, including the author of future versions. Additionally, this document is intended for readers with a technical background, such as interviewers or engineers reviewing the project, who wish to understand the design rationale and scope of implementation.

The current implementation focuses on desktop and browser-based applications for recording and calculating Scrabble scores. Advanced features such as real-time multiplayer gameplay, AI opponents and mobile support have been excluded from the current scope. These features may be addressed in later iterations of the project and are considered potential future enhancements.

2. Background and Goals

Problem:

Scrabble players typically record their scores on paper and calculate totals manually for each game, which is time-consuming and error prone. When using spreadsheets such as Excel, players must manually prepare calculation formulas and clear previous scores at the start of every game, resulting in additional setup effort and interruptions to gameplay.

Goal:

Develop an application that Scrabble players easily record, calculate and visualise their game scores. Key features include:

- Automate a calculation of score on app
- User-based historical score tracking
- Graph visualisation of score

Stakeholders:

Player, developer

3. Implementation Plan

The implementation of this app is divided into three steps.

The first implementation focuses on creating basic functions in a local environment. The second implementation involves transferring Java logic to Spring Boot to create an API. The app will be published and made available on mobile phones in the third implementation.

First Implementation:

Programming language	Java
Development environment	IntelliJ
UI framework	JavaFX
Architecture	MVC
Database	SQLite
Version control	GitHub

Second Implementation:

Programming language	Java, JavaScript
Development environment	IntelliJ, VS Code
UI framework	HTML, CSS, JavaScript
Architecture	Spring Boot (Controller/Service/Model) + REST API
Database	Spring Boot + SQLite
Version control	GitHub
Graph rendering	Chart.js on HTML
Hosting	local

Third Implementation:

Database	Firebase Firestore or Supabase
Hosting	Own domain

4. Functional Requirements

Must: Core functions for storing and calculating scores.

ID	User Story	Requirement Breakdown
1	As a user, I want the system to be able to read score that I input, so that I can record my score.	**Frontend:** A front-end interface must allow users to input their score as an integer.
		Backend: The backend must accept the score input sent from the frontend and validate it (e.g., type checking, business rules).
		The validated score should be stored in a database table, associated with the correct user and game session if applicable.
2	As a user, I want the system to automatically calculate the recorded scores, so that I don't have to do it manually.	**Backend:** The backend must calculate the total scores of all players using data from the score table.
		The calculation should be triggered automatically whenever new scores are added or existing scores are updated.
3	As a user, I want to be able to edit previously entered scores, so that I can correct any mistakes.	**Frontend:** A button or interface element must allow the user to select and edit a previously entered score
		Backend: The backend must update the corresponding score in the database table when a user edits it.
		The system must also recalculate the total score to reflect the updated value.
4	As a user, I want to see the rankings based on scores, so that I can understand the current standings.	**Frontend:** The frontend must display usernames and scores in descending order based on their ranking.
		Backend: The backend must generate a ranking by sorting users according to their total scores.
5	As a user, I want to see score progression within a single game, so that I can visually track how scores change turn by turn.	**Frontend:** Line graph must display each player's score progression over time.
		Backend: The backend should provide structured score data per player (e.g., by round or turn) to support line graph generation.
		The data must be formatted in a way that allows easy plotting on the frontend.
6	As a user, I want to view a list of my past games, including total scores and rankings per game, so that I can review my	**Frontend:** A dedicated page or section must be implemented to show a user's score and

	performance history.	ranking history.
		This page should display a table of total scores and ranking per game, sorted by date.
		Backend: The backend must store historical score data and ranking per user and in the database.
7	As a user, I want to be able to register or select a group, so that our game data can be saved and managed as a unit.	**Frontend:** A button or interface element must allow the user to select or register a new group with a set of fixed members.
		Group information (group name, member names) must be input during creation.
		Backend: The backend must store player information in the database.
		All future score data must be associated with the registered group.
8	As a user, I want to create a temporary group for one-time play, so that we can enjoy a game session without saving any data.	**Frontend:** A button or interface element must allow the user to create a temporary group.
		Member names should be input during creation. (“Player ”+number by default)
		Backend: The backend must support gameplay using temporary groups but should not store their score data persistently.
9	As a user, I want to change the play order in a new game, so that I can customize the sequence of turns.	**Frontend:** A button must allow the user to rearrange the order of players before the game starts.
		Backend This order must be used when initialising the score table and determining turn sequence.
10	As a user, I want the system to display the current score and ranking, so that I can stay engaged and enjoy the competition.	**Frontend:** The current score and ranking must be displayed in real-time during gameplay.
		A toggle should allow users to manually show or hide the score and ranking display.
		The system should have an option of hiding the score and ranking automatically once a certain number of pieces have been placed.
		Backend: The backend must maintain the current total scores for each player.
		Rankings must be dynamically generated based on those total scores.
25	As a user, I want the game to end automatically when all players score zero for three consecutive rounds, so that the game follows official Scrabble endgame rules.	**Frontend**: Display the current count of consecutive zero-score rounds.
		When the game ends due to consecutive zero-score rounds, display a clear game-over message explaining the reason.
		Backend:

		Track consecutive zero-score turns for each player.
		Detect when all players have scored zero for a number of consecutive turns equal to the number of players over three full rounds.
		Trigger a game-over event and notify the UI layer when the condition met.

Should: Reasonable functions for improving usability.

ID	User Story	Requirement Breakdown
11	As a user, I want to have an option to set a time limit, so that I can control the pace and intensity of the game.	**Frontend** A timer that the user set before the game starts must be displayed on screen during gameplay.
		When a player's time runs out, the system should prompt them to enter their score (e.g., with a ringing sound or alert).
		Backend: The backend must reset and start a countdown timer each time a player finishes their turn.
12	As a user, I want to view my past scores in a graph, so that I can analyse my performance trends over time.	**Frontend:** Frontend should display two time-series line graph of total scores and ranking per game.
		A toggle should allow users to switch between score graph and ranking graph
13	As a user, I want to compare past scores between multiple users, so that I can evaluate relative performance.	**Frontend:** A toggle must allow the user to select multiple players for comparison.
		Backend: The backend must provide score history data for multiple users, formatted for time-series comparison.
14	As a user, I want to be able to delete past score records, so that I can manage my data more cleanly.	**Frontend:** A delete button must be available to allow users to remove score history entries.
		The UI should confirm the deletion action (e.g., with a confirmation dialog) to prevent accidental loss.
		Backend: The backend must delete the specified score data from the database once the user confirms deletion.
15	As a user, I want to pause and resume a game in progress, so that I can continue playing without losing my progress.	**Frontend:** A pause button must be available during gameplay.
		When the game is paused, the UI must display options to resume or quit.
		If a timer is active, it must be visibly stopped or frozen during the pause state.
		Backend:

		If a timer is active, the backend must pause the countdown when the game is paused.
24	As a user, I want to enter a word and have its score calculated automatically according to Scrabble rules, so that I do not need to calculate scores manually.	**Frontend**: Provide an input field that allows the user to enter a word and submit it as a turn score.
		Represent each letter of the entered word as an individually selectable UI element and visually indicate which letters or word modifiers are currently applied.
		Provide controls to apply Double Letter/ Triple Letter/ Double Word/ Triple Word modifiers to selected letters.
		Provide a control to apply the 50-point bonus for using all tiles in a turn.
		Backend: Maintain a data set defining the point value for each Scrabble letter.
		When a letter modifier is applied, calculate the letter's score using the appropriate multiplier.
		When a word modifier is applied, apply the multiplier to the total word score after letter-level calculations.
		Include the 50-point bonus in the final score when selected.
		After a score is submitted, clear all word- and letter-related inputs and prepare the system for the next turn.

Could: Functions required to enhance the user experience with visualisation and intuitive usability.

ID	User Story	Requirement Breakdown
16	As a user, I want to record moves on the game board, so that the system can calculate special pieces and apply their effects to the score.	**Frontend:** The interface must include a Scrabble-style board that visually replicates the standard layout.
		When a user clicks on a square, an arrow should appear to indicate the input direction (right or down).
		After selecting a direction, the word input must be displayed on the board in that direction.
		Backend: The backend must receive the word data and compute the score according to Scrabble rules and the board layout.
17	As a user, I want the system to store board states from the past five games, so that I can review recent gameplay.	**Frontend:** The interface must allow users to view the Scrabble board from each of the past five games.
		Each board should visually replicate the original game state, including placed tiles and special squares.
		Backend:

		The backend must store the complete board state for each game in a restorable format
		Only the most recent five games per user need to be retained. Older data may be overwritten or archived as needed.
18	As a user, I want the application to be mobile-friendly, so that I can play comfortably on my smartphone.	Mobile-friendly design and system is required. (TBD)
19	As a user, I want the system to divide users into three authority groups, so that I can organise games with appropriate level of permission.	**Frontend:** The UI must indicate each user's authority level (e.g., owner, editor, viewer) within a group or session.
		Actions that exceed the user's authority should be disabled or hidden from the interface.
		Backend: The backend must define and enforce three authority levels: Owner, Editor, Viewer
		All operations must be checked against the user's authority before being processed. Unauthorized actions must be rejected.

Won't: Features with high development workload that will not be implemented this time.

ID	User Story	Requirement Breakdown
20	As a user, I want to share the game screen using an invitation code, so that I can easily play with others remotely.	**Frontend:** A button must allow the current user to generate and share an invitation code.
		Another interface must allow users on other devices to input the code and join the ongoing game.
		Once connected, the guest device should be navigated to the current game screen, where they can view and edit the shared content live.
		Backend: The backend must generate a unique invitation code linked to the current game session.
		When another user inputs the code, the backend must associate that session with their device.
		The backend must support live synchronisation of game data between devices.
21	As a user, I want multiple players to be able to enter their scores, so that gameplay can proceed smoothly without a dedicated scorekeeper.	**Frontend:** When a player is inputting a score, input fields on other connected devices must be temporarily disabled to prevent conflicts.
		The UI should clearly indicate which player is currently recording their score.
		Backend: The backend must support real-time

		synchronisation of score input status between devices.
22	As a user, I want the system to check whether a word is valid, so that only acceptable entries are used in the game.	**Frontend** When a player inputs a word, the UI must display a warning if the word is invalid, highlighting and blocking from being submitted.
		Backend: The backend must automatically validate inputted words using a Scrabble-approved dictionary.
23	As a user, I want to play against an AI opponent, so that I can enjoy the game even when no other players are available.	**Frontend:** A button must be provided to allow the user to start a game against an AI opponent. (TBD)

5. Non-Functional Requirements

Performance

- Score totals must be updated immediately after each score input.
- The application should respond to user input within a perceptible timeframe.

Persistence & Reliability

- Game and score data must be persisted using a local SQLite database.
- Stored data must remain available after the application is closed and restarted.

Maintainability

- The application must follow a layered architecture separating UI, service, and data access logic.
- Business logic must not depend on UI-specific frameworks.

Usability

- The score input process should be simple enough to be completed during gameplay without interrupting the flow.
- Player names and turn order must be clearly visible during score entry.

6. Screen Transitions

- Menu Scene

Role
Serves as the main entry point of the application and guides users to major sections.

Trigger	Next Scene
Select “Create Group”	Group Setup Scene
Select “Select Group”	Group Selection Scene
Select “Settings”	Setting Scene

- Group Setup Scene

Role
Allows users to define the number of players and configure player names.

Trigger	Next Scene
Select “Next”	Game Setup Scene
Select “Back”	Confirmation dialog displayed
Confirm “Discard changes”	Menu Scene
Cancel confirmation	Remain on Group Setup Scene

- Group Selection Scene

Role
Allows users to select a previously saved group.

Trigger	Next Scene
Select “Edit Group Info”	Group Setup Scene
Select “Next”	Game Setup Scene
Select “Back”	Menu Scene

- Game Setup Scene

Role
Allows users to configure game-specific settings before starting a game.

Trigger	Next Scene
Select “Start Game”	Score Input Scene
Select “Back”	Group Setup Scene

- Score Input Scene

Role
Allows users to input scores turn by turn during gameplay.

Trigger	Next Scene
Select “Finish Game”	Ranking Scene
Select “Pause”	Pause dialog displayed
Select “Quit Game” (from pause)	Menu Scene
Select “Back” (from pause)	Resume Score Input Scene (timer restarts if enabled)

- Ranking Scene

Role
Displays the final results of the game, including total scores and player rankings.

Trigger	Next Scene
Select “Back to Menu”	Menu Scene
Select “Chart”	Chart View Scene
Select “Another Game”	Game Setup Scene

- Chart View Scene

Role
Displays detailed score progression using charts and tables.

Trigger	Next Scene
Select “Back to Menu”	Menu Scene
Select “See Rankings”	Ranking Scene

- Setting Scene

Role
Allows users to manage application and data-related settings.

Trigger	Next Scene
(To be defined)	(To be defined)

7.Revisions

v.0.2.0 - Background of Reclassification

During the implementation planning phase, several advanced requirements (R16–R23) were reviewed and reorganised to better align with architectural concerns and development phases.

Originally, these requirements were classified using the MoSCoW method. However, further analysis revealed that grouping them solely by priority caused inconsistencies in terms of system design and implementation feasibility.

As a result, the requirements were reclassified based on their primary technical concerns:

- **Board-centric features** (R16, R17, R22), which involve Scrabble board representation, rule-based score calculation, and dictionary validation, were identified as advanced game logic extensions. These features introduce significant UI and domain complexity and are therefore excluded from the core scope of the first/second implementation. However,

- **Multi-user and collaboration features** (R19) were recognised as dependent on a web-based architecture and state-sharing mechanisms. These requirements are planned for the second implementation phase, where the system is migrated to Spring Boot with RESTful APIs.

- **Real-time screen sharing via invitation codes** (R20, R21) was explicitly excluded from the project scope due to its high implementation cost and reliance on live synchronisation between multiple devices.

- **Platform and experience extensions**, such as mobile-friendly design (R18) and AI opponent functionality (R23), were categorised as optional future enhancements and are not essential to the core scoring functionality.

These revisions do not change the fundamental goal of the project but clarify the scope and development strategy to ensure a feasible and coherent implementation plan.

Reclassified Requirements

ID	Original Classification	Planning Classification	Reclassified Group	
R16	Could	Could	Board-centric	Requires board representation and rule-based scoring
R17	Could	Could	Board-centric	Persistent board state storage
R18	Could	Won’t / Future	Platform Extension	Depends on web-based UI
R19	Could	Could	Multi-user	Requires user roles and access control
R20	Won’t	Won’t	Multi-user	High cost due to live synchronisation
R21	Won’t	Won’t	Multi-user	Real-time state management
R22	Won’t	Could	Board-centric	Dictionary integration required
R23	Won’t	Won’t / Future	Game Experience Extension	AI opponent

v.0.3.0 – Additional requirements

The additional functional requirement (R24) has been added to clarify the Scrabble-specific scoring behaviour.

Previously, Scrabble-related features were largely deferred as 'Could' or 'Won't' requirements, resulting in a primarily generic score calculator after the initial implementation.

R25 has also been added to enable users to access advanced gameplay with official rules.

New requirements:

- **Word-based score calculation** (R24)
- **Automatic Game Termination on Consecutive Zero-Score Rounds** (R25)

This revision improves the clarity of the application’s intended specialisation while keeping advanced Scrabble features out of the initial scope.